Solution: Palindromic Substrings

Problem Statement

Given a string, determine the number of palindromic substrings present in it.

A palindromic substring is a sequence of characters that reads the same forwards and backward. The substring can be of any length, including 1.

Example

1.
   - **Input**: "racecar"
   - **Expected Output**: 10
   - **Justification**: The palindromic substrings are "r", "a", "c", "e", "c", "a", "r", "cec", "aceca", "racecar".
2.
   - **Input**: "noon"
   - **Expected Output** 6
   - **Justification**: The palindromic substrings are "n", "o", "o", "n", "oo", "noon".
3.
   - **Input**: "apple"
   - **Expected Output**: 6
   - **Justification**: The palindromic substrings are "a", "p", "p", "l", "e", "pp".

Solution

The core idea behind the algorithm is to consider each character in the string as a potential center of a palindrome and then expand outwards from this center to identify all palindromic substrings.

By doing this for every character in the string, we can efficiently count all such substrings. This approach is based on the observation that every palindromic substring has a center (or two centers for even-length palindromes).

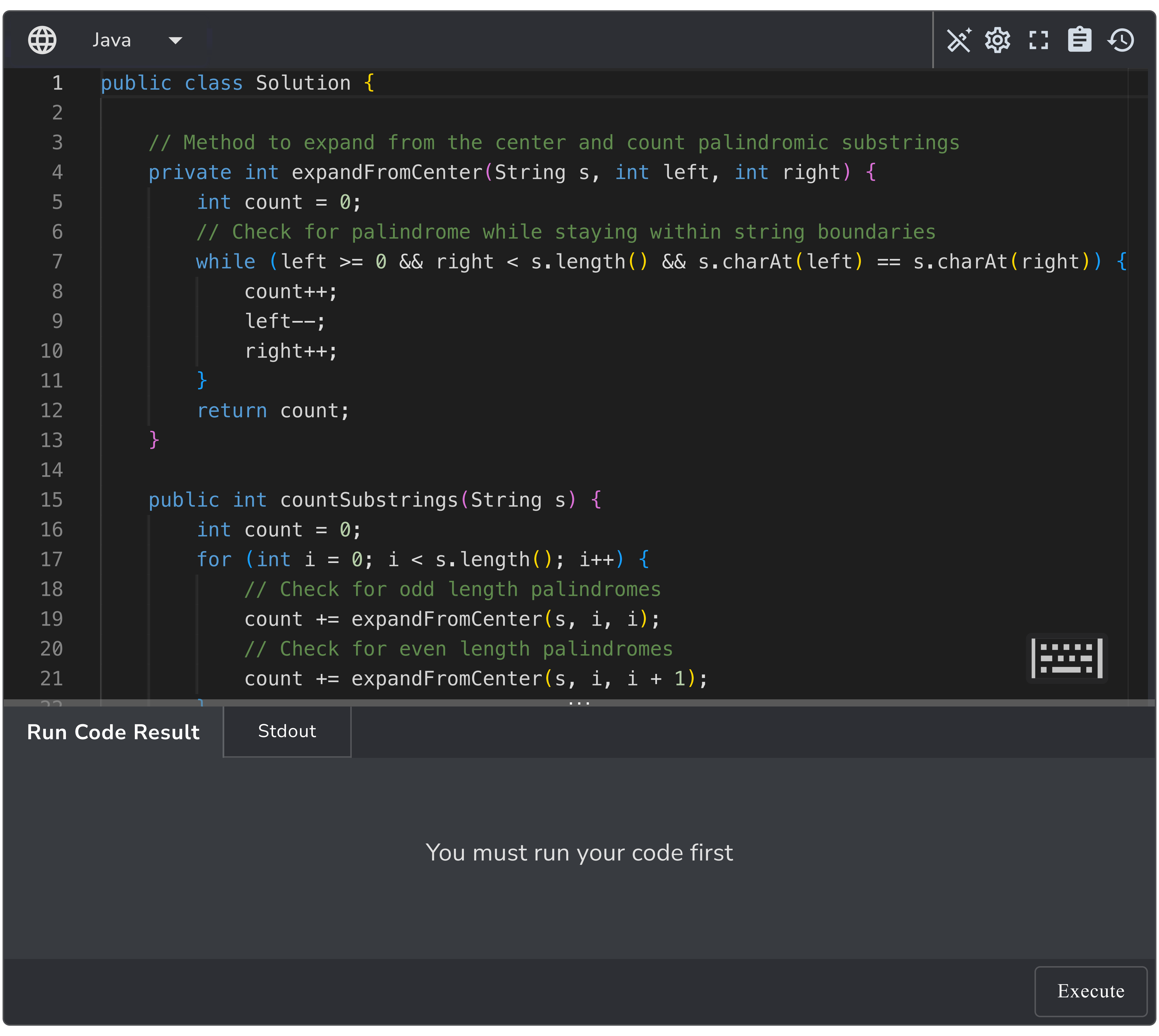
1. **Initialization:** Begin by initializing a counter to zero. This counter will be used to keep track of the number of palindromic substrings.
2. **Center Expansion:** For each character in the string, treat it as the center of a possible palindrome. There are two scenarios to consider: odd-length palindromes (with a single center) and even-length palindromes (with two centers). For each character, expand outwards and check for both scenarios.
3. **Palindrome Check:** As you expand outwards from the center, compare the characters. If they are the same, increment the counter. If they are different or if you've reached the boundary of the string, stop expanding.
4. **Result:** Once all characters have been treated as centers and all possible expansions have been checked, the counter will hold the total number of palindromic substrings.

Algorithm Walkthrough

Using the input "noon":

- Start with the first character "n":
  - Treat it as the center of an odd-length palindrome. No expansion is possible.
  - Treat it as the left center of an even-length palindrome. The right center would be "o". Since "n" and "o" are different, no expansion is possible.
- Move to the second character "o":
  - Treat it as the center of an odd-length palindrome. No expansion is possible.
  - Treat it as the left center of an even-length palindrome. The right center is also "o". Increment the counter.
- Continue this process for each character in the string.
- The final count is 7.

Code



```
public class Solution {

    // Method to expand from the center and count palindromic substrings
    private int expandFromCenter(String s, int left, int right) {
        int count = 0;
        // Check for palindrome while staying within string boundaries
        while (left >= 0 && right < s.length() && s.charAt(left) == s.charAt(right)) {
            count++;
            left--;
            right++;
        }
        return count;
    }

    public int countSubstrings(String s) {
        int count = 0;
        for (int i = 0; i < s.length(); i++) {
            // Check for odd length palindromes
            count += expandFromCenter(s, i, i);
            // Check for even length palindromes
            count += expandFromCenter(s, i, i + 1);
```

```
You must run your code first
```

Complexity Analysis:

- **Time Complexity**: O(n^2). For each character in the string, we might expand outwards up to n times.
- **Space Complexity**: O(1). We are not using any additional data structures that scale with the input size.

← Previous
Palindromic Substrings (Medium)
Next →
Remove Nth Node From End Of List (Medium)
☐ Mark as Completed